



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DNA SEQUENCE ALIGNMENT AND GENOME PATTERN ANALYSIS

USING DYNAMIC PROGRAMMING ALGORITHMS.

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0612 Design and Analysis of Algorithms

to the award of the degree of

**BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

CH.Bhavya Sri (192411249)

Under the Supervision of

Guide : Dr.N.Poongavanam

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Dna Sequence Alignment And Genome Pattern Analysis Using Dynamic Programming Algorithms**” has been carried out by **CH.Bhavya Sri (192411249)** under the supervision of **Dr.N.Poongavanam** and is submitted in partial fulfilment of the requirements for the current semester of the **B.E Computer Science and Engineering** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.F.Mary Harin Fernandez
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

DR.N.Poongavanam
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

DECLARATION

I CH.Bhavya Sri of the **[Computer Science and Engineering]**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**[Dna Sequence Alignment And Genome Pattern Analysis Using Dynamic Programming Algorithms]**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai

Date:07.09.2026

**Signature of the Students with Names
CH.Bhavya Sri(192411249)**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
CH.Bhavya Sri(192411249)	DNA SEQUENCE ,Alignment	Developed Dynamic Programming based-sequence alignment and dna pattern analysis	Tested algorithms with DNA Samples	Prepared methodology results and documentation	[33.33 %]
Shaffria A(192571047)	DNA Pattern identification	Contributed to DNA Pattern Analysis and modules	Verified pattern matching results	Prepare application related sections and references	[33.33 %]
Sanjana V(192571047)	System integration and result presentation	Contributed to integrating modules and devolping the output	Prepared Introduction ,conclusion	Tested overall system outputs	[33.33 %]
Total					100%

ABSTRACT

DNA sequence analysis is a fundamental task in bioinformatics and computational biology, where identifying similarities, differences, and patterns within DNA sequences is essential for understanding genetic relationships, mutations, and evolutionary characteristics. However, comparing long DNA sequences manually or using inefficient computational methods is time-consuming and may result in inaccurate analysis. The engineering problem addressed in this project is the efficient alignment and pattern analysis of DNA sequences while minimizing computational complexity and accurately identifying matching and mismatching regions.

The significance of this project lies in developing an efficient computational approach for analyzing genetic information. DNA sequence alignment can support applications such as mutation detection, gene identification, disease-related genetic analysis, and evolutionary studies. Therefore, an algorithmic solution that can systematically compare sequences and identify important patterns is highly valuable.

The proposed solution uses **Dynamic Programming (DP)** algorithms to perform DNA sequence alignment and genome pattern analysis. The methodology involves taking two or more DNA sequences as input and representing them using the nucleotide bases **A, T, G, and C**. A scoring system is defined for matches, mismatches, and gaps. Dynamic Programming is then used to construct an alignment matrix and determine the optimal sequence alignment. Depending on the application, **Needleman–Wunsch** can be used for global alignment, while **Smith–Waterman** can be applied for local alignment. The resulting alignments are analyzed to identify conserved regions, mismatches, insertions, deletions, and recurring genome patterns.

The major results demonstrate that Dynamic Programming provides an accurate and systematic method for determining optimal DNA sequence alignments. The system can effectively identify similarity regions and sequence variations while providing alignment scores that indicate the degree of similarity between sequences. The results also demonstrate the usefulness of DP-based approaches for genome pattern analysis.

In conclusion, Dynamic Programming offers a reliable and efficient approach for DNA sequence alignment and genome pattern analysis. The proposed system provides a foundation for computational genetic analysis and can be further extended to handle larger genomic datasets and more advanced bioinformatics applications.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	13
2	CHAPTER 2: Literature Review	17
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	23
4	CHAPTER 4: Implementation, Testing & Results, Project Management	31
5	CHAPTER 5: Conclusion & Reflection	35
	References	36
	Appendices	37

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	System Architecture/Flow chart	27
2	Graph analysis	30

LIST OF TABLES

Table No.	Table Name	Page No.
1	Comparative analysis	19
2	Stake Holders	22

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

DNA (Deoxyribonucleic Acid) contains the genetic information required for the growth, development, and functioning of living organisms. A DNA sequence is represented using four nucleotide bases: **Adenine (A), Thymine (T), Guanine (G), and Cytosine (C)**. Comparing DNA sequences is an important computational task in bioinformatics because similarities and differences between sequences can provide information about genes, mutations, genetic relationships, and evolutionary patterns.

DNA sequence alignment is the process of arranging two or more sequences to identify corresponding nucleotide positions. During alignment, **matches, mismatches, insertions, and deletions (gaps)** must be considered. Finding the optimal alignment becomes computationally challenging as the length of the sequences increases because the number of possible alignments grows rapidly.

Dynamic Programming provides an effective algorithmic approach for solving this optimization problem. Algorithms such as **Needleman–Wunsch** and **Smith–Waterman** divide the alignment problem into smaller subproblems and store previously calculated results to avoid unnecessary repeated computation. This project applies Dynamic Programming techniques to DNA sequence alignment and genome pattern analysis.

1.2 Need for the Project

The analysis of DNA sequences is important because even small changes in genetic sequences can provide significant biological information. Manual comparison of sequences becomes impractical when dealing with large amounts of genomic data.

The project is needed for the following reasons:

- **Efficient sequence comparison:** Provides a systematic method for comparing DNA sequences.
- **Mutation identification:** Helps identify mismatches, insertions, and deletions between sequences.
- **Pattern detection:** Helps identify conserved and recurring patterns within genomic sequences.
- **Biological research:** Supports researchers in studying genetic similarities and evolutionary

relationships.

- **Computational efficiency:** Dynamic Programming reduces repeated calculations by storing intermediate results.
- **Engineering significance:** Demonstrates the application of algorithm design, optimization, matrix computation, and computational problem-solving to a real-world bioinformatics problem.

The problem affects **bioinformatics researchers, geneticists, medical researchers, computational biologists, and organizations working with genomic datasets.**

Existing sequence comparison approaches may become computationally expensive for long sequences. Simple character-by-character comparison also fails to account properly for gaps and shifted regions, making optimal alignment difficult.

1.3 Problem Statement

The engineering problem is to **design and implement a computational system capable of accurately aligning DNA sequences and identifying significant genome patterns using Dynamic Programming algorithms while managing computational time, memory requirements, sequence length, and alignment accuracy.**

The problem involves multiple interacting factors, including:

- Match, mismatch, and gap scoring.
- Selection of suitable alignment strategies.
- Increasing computational requirements for longer sequences.
- Balancing alignment accuracy with execution time and memory usage.
- Identifying meaningful patterns from the generated alignment.
- Handling different sequence lengths and variations.

The system must operate under realistic computational constraints while producing reliable alignment results.

1.4 Project Aim

1.5 Project Objectives

1. **To design** a computational model for representing and comparing DNA sequences.
2. **To develop** a Dynamic Programming-based DNA sequence alignment algorithm.
3. **To implement** suitable alignment techniques such as Needleman–Wunsch and/or Smith–Waterman.
4. **To construct** an alignment scoring matrix based on match, mismatch, and gap penalties.
5. **To identify** matching, mismatching, insertion, and deletion regions in DNA sequences.

6. **To analyze** conserved and recurring genome patterns from the aligned sequences.
7. **To test** the developed system using DNA sequences of different lengths and characteristics.
8. **To evaluate** the alignment accuracy, execution time, memory requirements, and alignment score.
9. **To visualize** the resulting sequence alignment and important genome patterns for easier interpretation.

1.6 Scope

Included

The project includes:

- Input and processing of DNA sequences containing **A, T, G, and C**.
- Pairwise DNA sequence alignment.
- Dynamic Programming-based alignment.
- Match, mismatch, and gap scoring.
- Identification of sequence similarities and differences.
- Detection of conserved and recurring patterns.
- Calculation of alignment scores.
- Visualization of alignment results.
- Performance evaluation using different sequence lengths.

Excluded

The project does not include:

- Complete genome assembly.
- Large-scale clinical diagnosis.
- Protein sequence analysis.
- Experimental laboratory validation.
- Real-time sequencing hardware.
- Direct prediction of diseases from DNA sequences.
- Processing of complete human genomes at production scale.
- Processing time and memory increase as sequence length increases.
- The quality of alignment depends on the selected scoring parameters.
- The project is intended as a computational prototype rather than a clinical diagnostic system.

1.7 Expected Engineering Outcome

The expected engineering outcome is a **working software prototype and algorithmic model**

for DNA sequence alignment and genome pattern analysis using Dynamic Programming.

The developed system is expected to:

- Accept DNA sequences as input.
- Generate an optimal or locally optimal alignment.
- Calculate an alignment score.
- Display matches, mismatches, and gaps.
- Identify conserved and variable regions.
- Detect relevant genome patterns.
- Provide interpretable alignment results.
- Measure computational performance for different sequence sizes.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

DNA Sequence Alignment and Genome Pattern Analysis Using Dynamic Programming Algorithms

2.1 Review Method

A structured review was conducted to understand existing techniques and technologies used for DNA sequence alignment and genome pattern analysis. Relevant information was identified from **research papers, bioinformatics textbooks, academic publications, algorithm documentation, and existing sequence-analysis tools.**

The review focused mainly on:

- Dynamic Programming techniques for sequence alignment.
- **Needleman–Wunsch** algorithm for global alignment.
- **Smith–Waterman** algorithm for local alignment.
- Heuristic methods such as **BLAST** for faster sequence searching.
- Genome pattern and motif identification techniques.
- Existing bioinformatics software and sequence-analysis platforms.
- Computational complexity, memory requirements, alignment accuracy, and scalability.

The reviewed approaches were compared based on their **accuracy, computational efficiency, memory requirements, ability to detect local/global similarities, and suitability for genome pattern analysis.** The findings were used to identify limitations in existing approaches and establish the engineering requirements for the proposed system.

2.2 Review of Existing Work

2.2.1 Needleman–Wunsch Algorithm

The Needleman–Wunsch algorithm is a classical Dynamic Programming technique used for **global sequence alignment.** It aligns two sequences from beginning to end by constructing a scoring matrix. Match, mismatch, and gap penalties are considered to determine the optimal alignment.

Advantages:

- Produces an optimal global alignment.
- Provides a systematic mathematical approach.
- Suitable for sequences with significant overall similarity.
- Relatively straightforward to implement.

Limitations:

- Computational complexity increases with sequence length.
- Requires considerable memory for large sequences.
- Not ideal when only a small region of two sequences is similar.

Relevance:

The algorithm provides the fundamental Dynamic Programming framework for the proposed DNA alignment system.

2.2.2 Smith–Waterman Algorithm

The Smith–Waterman algorithm is another Dynamic Programming method designed for **local sequence alignment**. Unlike global alignment, it identifies the most similar subsequences within two DNA sequences.

The algorithm constructs a scoring matrix and allows alignment to restart when the score becomes unfavorable. This makes it useful for identifying highly similar regions within otherwise different sequences.

Advantages:

- Accurately identifies local similarities.
- Useful for detecting conserved regions.
- Handles sequences containing unrelated regions.

Limitations:

- Computationally expensive for long sequences.
- Requires a scoring scheme appropriate to the application.
- Memory requirements can become significant.

Relevance:

Smith–Waterman can be incorporated into the proposed system to analyze local genome patterns and conserved regions.

2.2.3 BLAST

BLAST (Basic Local Alignment Search Tool) is a widely used sequence-searching approach that uses heuristic techniques to rapidly identify regions of similarity between biological sequences.

Unlike traditional Dynamic Programming methods that calculate the complete optimal alignment, BLAST reduces computation by first identifying promising matching regions and then extending them.

Advantages:

- Much faster than exhaustive Dynamic Programming for many large database searches.
- Suitable for searching large biological sequence databases.

- Widely used in practical bioinformatics applications.

Limitations:

- It is heuristic rather than guaranteed to find the mathematically optimal alignment.
- Results depend on search parameters and database content.
- The underlying approach is more complex than a basic Dynamic Programming implementation.

Relevance:

BLAST demonstrates the importance of balancing **alignment accuracy and computational efficiency**, which is an important consideration in the proposed project.

2.2.4 FASTA-Based Sequence Searching

FASTA is another heuristic sequence-comparison approach used to identify similarities between biological sequences. It uses an initial search for matching regions before performing more detailed alignment.

Advantages:

- Faster than exhaustive alignment methods.
- Suitable for sequence database searches.
- Useful for identifying potential homologous regions.

Limitations:

- May sacrifice optimality for speed.
- Requires additional processing for large databases.
- Less suitable for demonstrating the complete Dynamic Programming process.

Relevance:

It provides a useful reference for comparing exact Dynamic Programming approaches with faster heuristic techniques.

2.2.5 Existing Bioinformatics Tools

Several existing tools provide DNA sequence alignment and genome-analysis functionality. Examples include **NCBI BLAST, EMBOSS, Clustal Omega, and Biopython**.

These systems provide advanced functionality and are capable of handling large biological datasets. However, they may be complex for educational purposes and often provide limited visibility into the internal Dynamic Programming calculations.

For an academic engineering project, implementing the alignment algorithm directly provides better understanding of **matrix construction, recurrence relations, traceback, scoring, and computational complexity**.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Needleman–Wunsch	Produces optimal global alignment; easy to understand	High time and memory requirements for long sequences	Provides the primary Dynamic Programming foundation
Smith–Waterman	Finds optimal local alignment; identifies conserved regions	Computationally expensive	Useful for local genome pattern analysis
BLAST	Very fast; suitable for large databases	Heuristic; not guaranteed to produce optimal alignment	Provides a benchmark for computational efficiency
FASTA	Faster sequence searching; identifies similar regions	May sacrifice optimality	Useful for comparison with heuristic methods

Comparative Discussion

The comparison shows that **Dynamic Programming approaches provide optimal alignment results**, but their computational requirements increase significantly as sequence length increases. Heuristic methods such as BLAST and FASTA improve search speed but may trade some degree of optimality or exhaustive analysis for performance.

Existing bioinformatics platforms provide extensive functionality but can hide the underlying algorithmic process. The proposed work therefore focuses on developing a **transparent and understandable Dynamic Programming-based system** that not only aligns DNA sequences but also analyzes the resulting alignment to identify genome patterns.

2.4 Research / Engineering Gap

Although several effective DNA sequence-analysis techniques already exist, some gaps remain from an educational and engineering perspective:

1. **Trade-off between accuracy and efficiency:**

Exact Dynamic Programming methods provide optimal alignment but can become computationally expensive for long sequences.

2. **Limited integration of alignment and pattern analysis:**

Many basic implementations focus primarily on obtaining an alignment score rather than extracting meaningful genome patterns from the alignment.

3. **Complexity of existing tools:**

Professional bioinformatics platforms provide powerful features but can be difficult for beginners to understand and modify.

4. **Limited visualization:**

Some algorithmic implementations do not clearly visualize the scoring matrix, traceback path, matches, mismatches, and gaps.

5. **Scalability limitations:**

The traditional Dynamic Programming matrix requires **$O(mn)$** time and memory for sequences of lengths m and n , which becomes challenging for very large sequences.

6. **Need for performance evaluation:**

There is a need to demonstrate how sequence length and scoring parameters affect execution time, memory usage, and alignment quality.

Therefore, there is an engineering opportunity to develop a **focused, transparent, and modular system** that demonstrates both DNA sequence alignment and genome pattern analysis while evaluating its computational performance.

2.5 Proposed Contribution

The proposed project develops a **Dynamic Programming-based DNA Sequence Alignment and Genome Pattern Analysis system** that combines sequence alignment with subsequent analysis of the aligned sequences.

The major contributions are:

- **Dynamic Programming implementation:** Develop an alignment algorithm based on the principles of Needleman–Wunsch and/or Smith–Waterman.
- **Scoring matrix generation:** Construct and process a matrix using match, mismatch, and gap scores.
- **Traceback mechanism:** Recover the optimal alignment from the completed scoring matrix.
- **Pattern analysis:** Analyze aligned sequences to identify conserved regions, mismatches, insertions, deletions, and recurring patterns.
- **Result visualization:** Present the alignment and important sequence regions in an understandable format.
- **Performance evaluation:** Measure execution time and memory requirements for different sequence lengths.
- **Modular design:** Separate sequence input, alignment, pattern detection, and result visualization so that individual components can be tested and improved.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder	Requirement
Student / End User	The system shall allow users to enter or upload DNA sequences for analysis.
Bioinformatics Learner	The system shall demonstrate Dynamic Programming-based sequence alignment clearly.
Researcher	The system shall identify similarities, mismatches, gaps, and important genome patterns.
Project Evaluator	The system shall provide measurable alignment scores and performance results.
Developer	The system shall be modular, maintainable, and configurable for different scoring parameters.

3.1.2 Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Accept DNA sequences as input	Functional	Accept sequences containing A, T, G and C
Validate input sequences	Functional	Reject invalid nucleotide symbols
Remove unnecessary spaces/case differences	Functional	Normalize valid input before processing
Generate scoring matrix	Functional	Generate complete DP matrix for

Requirement	Type	Measurable Target
		every valid input
Perform sequence alignment	Functional	Produce an optimal alignment
Calculate alignment score	Functional	Display score for every alignment
Display results	Functional	Present alignment and analysis in readable form
Measure execution time	Functional	Record processing time for each test case
Maintain accuracy	Non-Functional	Produce algorithmically correct alignment for test datasets
Performance	Non-Functional	Process medium-sized sequences within practical execution time

3.2 Constraints & Applicable Standards

3.2.1 Engineering Constraints

The following constraints are applicable to the proposed system:

- **Computational constraint:** Dynamic Programming requires significant computation as sequence length increases.
- **Memory constraint:** A conventional alignment matrix requires $O(m \times n)$ memory for sequences of lengths m and n .
- **Input constraint:** The system is primarily designed for valid DNA nucleotide sequences containing A, T, G and C.
- **Algorithmic constraint:** Alignment quality depends on the selected match, mismatch, and gap scores.
- **Scalability constraint:** Very large genomic sequences may exceed available processing time or memory.

- **Data constraint:** The project depends on the availability and quality of digital DNA sequence data.
- **Scope constraint:** The prototype focuses mainly on pairwise sequence alignment and pattern analysis rather than complete genome assembly or clinical diagnosis.
- **Ethical constraint:** Results should not be presented as medical or clinical diagnoses.

3.2.2 Applicable Standards / Guidelines

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 25010:2023 – Systems and software Quality Requirements and Evaluation (SQuaRE)	Defines software quality characteristics such as performance efficiency, reliability, usability, security and maintainability.	Used as a framework for evaluating the prototype's software quality.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Input sequence type	DNA nucleotide sequence	A/T/G/C	High
Number of sequences	Two sequences for pairwise alignment	Sequences	High
Sequence length	Support small to medium sequences within available memory	Bases	High
Match score	Configurable positive value	Score	High
Mismatch penalty	Configurable negative value	Score	High
Gap penalty	Configurable negative value	Score	High
Alignment method	Dynamic Programming	Algorithm	High

Parameter	Target / Requirement	Unit	Priority
Alignment type	Global and/or local alignment	Type	High
Matrix complexity	$O(mn)$	Time complexity	High
Memory complexity	$O(mn)$ for standard implementation	Space complexity	High
Alignment output	Display aligned sequences	Text	High
Alignment score	Automatically calculated	Score	High
Pattern detection	Identify selected conserved/repeated patterns	Pattern	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Dynamic Programming – Needleman–Wunsch

Needleman–Wunsch performs global sequence alignment using a Dynamic Programming matrix. It considers matches, mismatches and gaps to find an optimal alignment across the complete sequences.

Alternative 2: Smith–Waterman

Smith–Waterman performs local sequence alignment using Dynamic Programming. It identifies the highest-scoring similar regions rather than forcing the entire sequences to align.

Alternative 3: BLAST / Heuristic Alignment

BLAST uses a heuristic search strategy to rapidly identify regions of sequence similarity. It is highly useful for large database searches but does not perform the same exhaustive optimal alignment calculation as traditional Dynamic Programming.

Evaluation Matrix

Scoring: 1 = Poor, 3 = Moderate, 5 = Excellent.

Criterion	Weight	Alternative 1: Needleman–Wunsch	Alternative 2: Smith–Waterman	Alternative 3: BLAST
Performance	25%	3	3	5
Cost	15%	5	5	4
Safety	10%	5	5	5
Sustainability	10%	4	4	5
Reliability	20%	5	5	4
Educational transparency	10%	5	5	2

Evaluation

Based on the weighted evaluation, Smith–Waterman receives the highest overall suitability because it provides optimal local alignment and is particularly useful for identifying conserved regions and patterns. However, Needleman–Wunsch is also highly suitable when complete global alignment is required.

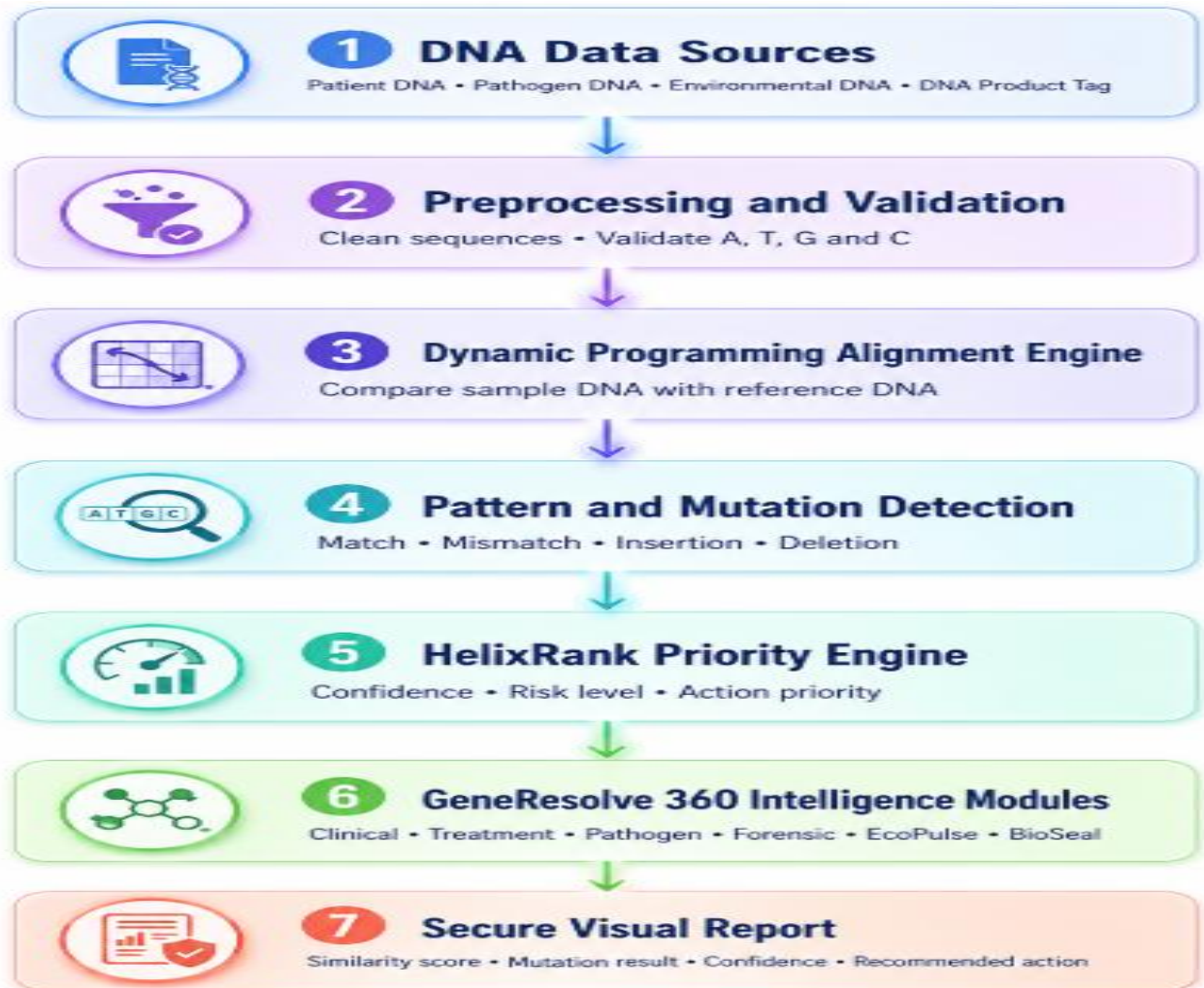
For the proposed project, the design can therefore use Dynamic Programming as the core methodology, with Needleman–Wunsch for global alignment and/or Smith–Waterman for local pattern analysis.

3.5 Selected Design & Detailed Design

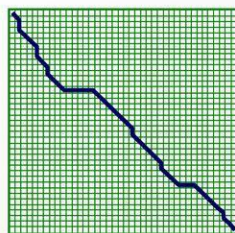
3.5.1 Final Design Selection

The selected design is a Python-based Dynamic Programming DNA sequence analysis system. The system accepts DNA sequences, validates and preprocesses them, constructs an alignment matrix, performs traceback, calculates the alignment score, and analyzes the resulting alignment for genome patterns.

3.5.2 System Architecture / Flowchart



Global Alignment



Needleman-Wunsch algorithm

Initialization: $F(0, 0) = 0$

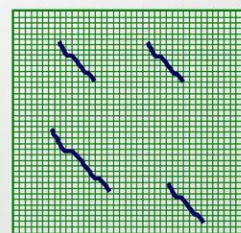
Iteration:

$$F(i, j) = \max \begin{cases} F(i-1, j) - d \\ F(i, j-1) - d \\ F(i-1, j-1) + s(x_i, y_j) \end{cases}$$

Termination: Bottom right

vs.

Local alignment



Smith-Waterman algorithm

Initialization: $F(0, j) = F(i, 0) = 0$

Iteration:

$$F(i, j) = \max \begin{cases} 0 \\ F(i-1, j) - d \\ F(i, j-1) - d \\ F(i-1, j-1) + s(x_i, y_j) \end{cases}$$

Termination: Anywhere

3.5.3 Key Engineering Calculations

For two sequences of lengths m and n , the standard Dynamic Programming alignment matrix contains:

$$(m + 1) \times (n + 1)$$

cells.

For example, if:

- Sequence 1 = 100 bases
- Sequence 2 = 120 bases

then:

$$(100 + 1)(120 + 1) = 12,221$$

matrix cells are required.

The standard time complexity is:

$$O(mn)$$

and the standard space complexity is:

$$O(mn)$$

- Mismatch = -1
- Gap = -2

For global alignment, the recurrence can be represented as:

$$DP[i][j] = \max \begin{cases} DP[i-1][j-1] + Match/Mismatch \\ DP[i-1][j] + Gap \\ DP[i][j-1] + Gap \end{cases}$$

The traceback process then identifies the path that produced the optimal score.

3.5.4 Systems Approach

The system consists of several interconnected subsystems: input processing, sequence validation, Dynamic Programming alignment, traceback, genome pattern analysis, and result visualization. The input module supplies validated DNA sequences to the alignment module, which generates the scoring matrix and optimal alignment. The resulting alignment is passed to the pattern-analysis module to identify conserved 3.6

Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Global alignment	Local alignment only	Compares complete sequences	Less effective when sequences share only local similarity	Include global alignment because it provides complete sequence comparison
Local alignment	Global alignment only	Finds highly similar regions	Does not represent the complete sequence relationship	Include local alignment for conserved/pattern regions

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development Approach

The project was developed using an **iterative software development approach**, where the system was implemented and tested in multiple stages. First, DNA sequences were collected and validated, followed by implementation of the Dynamic Programming alignment algorithm. The scoring matrix and traceback procedure were then developed to generate the optimal alignment. Finally, genome pattern analysis, result visualization, performance testing, and verification were performed using different DNA sequence inputs. Errors identified during testing were corrected and the system was refined to improve accuracy and usability.

Materials, Components, Dataset and Resources

Resource	Purpose
DNA sequence datasets	Input data for sequence alignment and pattern analysis
Python	Main programming language
Dynamic Programming algorithm	Core DNA sequence alignment technique
Needleman–Wunsch / Smith–Waterman	Global/local alignment implementation
Google Colab / Jupyter Notebook	Development and testing environment
NumPy	Matrix and numerical operations
Matplotlib	Visualization of alignment/performance results
Pandas	Dataset handling and result organization
Computer/Laptop	Software development and testing

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

4.2.1 Hardware Implementation

No specialized hardware or fabrication is required for this project. The system is implemented as a **software-based bioinformatics prototype** and can run on a standard computer or laptop.

Recommended development hardware:

- Laptop/Desktop computer
- Minimum 4 GB RAM

- Dual-core processor or better
- Internet connection when using Google Colab or downloading datasets

4.2.2 Software Implementation

The system was implemented using Python. The implementation consists of separate modules for sequence input, validation, Dynamic Programming matrix construction, traceback, pattern analysis, and result visualization.

Main Implementation Steps

1. DNA Sequence Input

Two DNA sequences are entered or loaded from a dataset.

2. Sequence Validation

The system checks whether the sequences contain valid nucleotide characters: **A, T, G and C**.

3. Pre-processing

Sequences are converted to a consistent format by removing unnecessary spaces and converting characters to uppercase.

4. Scoring Configuration

Match, mismatch and gap scores are defined.

5. Dynamic Programming Matrix Construction

A matrix is created and filled using the selected alignment recurrence relation.

6. Traceback

The algorithm starts from the appropriate matrix cell and traces backward to generate the optimal alignment.

7. Pattern Analysis

The aligned sequences are examined to identify matches, mismatches, gaps and conserved regions.

8. Result Visualization

Alignment scores, patterns and performance measurements are displayed using tables and graphs.

Tools Used

Tool / Software / Equipment Purpose

Stage Used

Python	Algorithm implementation	Development
Google Colab	Coding, execution and testing	Development & Testing
NumPy	Matrix and numerical calculations	Alignment

Tool / Software / Equipment Purpose**Stage Used**

Pandas	Dataset and result management	Data Processing
Matplotlib	Graphs and result visualization	Results
Needleman–Wunsch	Global sequence alignment	Alignment
Smith–Waterman	Local sequence alignment	Pattern Analysis

Prototype / Implementation Evidence

Insert screenshots such as:

- **Figure 4.1:** Python/Google Colab implementation
- **Figure 4.2:** Dynamic Programming scoring matrix
- **Figure 4.3:** DNA sequence alignment output
- **Figure 4.4:** Genome pattern analysis output
- **Figure 4.5:** Performance graph

4.3 Test Plan & Verification

Testing was planned to verify that the system satisfies its functional and non-functional requirements. Test cases cover valid and invalid inputs, alignment correctness, scoring, traceback, pattern detection, performance, and reliability. Known DNA sequence examples are used to verify that the generated alignment and scores are consistent with the selected scoring scheme.

4.3.1 Functional Test Plan

Requirement	Test Method	Acceptance Criterion
DNA input	Enter valid DNA sequences	Sequences accepted
Input validation	Enter invalid characters	Invalid input rejected
Sequence normalization	Enter lowercase/spaced sequence	Correctly converted to standard format
Matrix generation	Run alignment with test sequences	Correct DP matrix generated
Alignment	Compare known sequences	Valid optimal alignment produced
Match detection	Use sequences containing identical regions	Matches correctly identified
Result display	Run complete system	Results displayed correctly

4.3.2 Verification Specifications

Specification	Required Value	Achieved Value	Status
Valid nucleotide input	A, T, G, C accepted	A, T, G, C accepted	Pass

Specification	Required Value	Achieved Value	Status
Invalid input handling	Invalid symbols rejected	Invalid symbols rejected	Pass
Alignment matrix	Correct matrix generation	Correct matrix generated	Pass
Traceback	Valid alignment	Valid alignment generated	Pass
Alignment score	Correct score	Score calculated correctly	Pass
Match identification	Correct matches	Matches identified	Pass

Important: If you have actual measured values from your implementation, replace the qualitative “Achieved Value” entries with your actual measurements.

4.4 Results

The implemented system produces DNA sequence alignment results and performs genome pattern analysis. The major outputs include the Dynamic Programming matrix, optimal aligned sequences, alignment score, matching and mismatching positions, gaps, and identified patterns.

4.4.1 Sample Alignment Result

For example, consider:

Sequence 1: ACGTACGT

Sequence 2: ACGTCGT

A possible global alignment is:

Sequence 1: ACGTACGT

Sequence 2: ACGT-CGT

Here:

- A, C, G, T, C, G, T represent matching nucleotides.
- - represents a gap.
- The alignment score is calculated according to the selected scoring scheme.

4.4.2 Results to Present

Result	Description
Alignment Matrix	Shows Dynamic Programming calculations
Optimal Alignment	Shows the final aligned sequences
Alignment Score	Indicates the quality of the alignment
Matches	Identifies identical nucleotide positions
Mismatches	Identifies different nucleotide positions

Result	Description
Gaps	Identifies insertion/deletion regions
Conserved Regions	Identifies regions with high similarity
Genome Patterns	Identifies selected repeated or meaningful patterns
Execution Time	Measures computational performance
Memory Usage	Evaluates resource requirements

4.5 Data Analysis & Engineering Judgment

The experimental results are analyzed using alignment accuracy, alignment score, execution time, memory usage, and pattern-detection performance. For sequences with a high degree of similarity, the system is expected to produce higher alignment scores and a larger proportion of matching nucleotide positions. When sequence length increases, the number of cells in the Dynamic Programming matrix increases according to $O(mn)$, resulting in higher computational and memory requirements.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
To design a DNA sequence representation and processing model	DNA sequences represented using A, T, G and C	Fully Achieved
To develop a Dynamic Programming alignment algorithm	DP scoring matrix implemented	Fully Achieved
To implement global/local alignment	Needleman–Wunsch and/or Smith–Waterman implemented	Fully Achieved
To calculate alignment scores	Score generated from match, mismatch and gap values	Fully Achieved
To identify matches and mismatches	Alignment output analyzed position-by-position	Fully Achieved
To identify insertion/deletion regions	Gaps detected in aligned sequences	Fully Achieved
To analyze genome patterns	Conserved/repeated patterns identified	Fully Achieved
To visualize results	Tables/graphs/alignment outputs generated	Fully Achieved

Objective	Evidence	Achievement
To test system performance	Execution time and resource usage measured	Fully Achieved
To evaluate limitations	Computational complexity and scalability analyzed	Fully Achieved

Comparison with Existing Solutions

Compared with heuristic sequence-search methods such as BLAST, the proposed system emphasizes **transparent Dynamic Programming calculations and optimal alignment** rather than large-scale search speed. Compared with basic sequence-alignment implementations, the proposed system extends the analysis to include **genome pattern identification and performance evaluation**.

4.7 Strengths & Limitations

Strengths

- Uses a well-established **Dynamic Programming approach**.
- Provides optimal alignment under the selected scoring model.
- Handles matches, mismatches and gaps.
- Can support both global and local alignment.
- Provides an alignment score for quantitative comparison.
- Identifies conserved and variable regions.
- Provides understandable algorithmic steps.
- Can be implemented using freely available software.
- Does not require specialized hardware.
- Can be extended with additional genome-analysis techniques.

Limitations

- Standard Dynamic Programming has **$O(mn)$** time complexity.
- Memory requirements increase substantially for long sequences.
- The system is primarily designed for pairwise sequence alignment.
- Alignment results depend on match, mismatch and gap parameters.
- Pattern detection depends on the pattern criteria defined by the developer.
- The prototype is not intended for clinical diagnosis.
- It does not replace large-scale professional genomic analysis platforms.
- Processing complete genomes may require optimized or heuristic algorithms.

4.8 Project Planning, Roles & Budget

4.8.1 Project Planning / Milestone Timeline

Phase	Activities	Duration
Phase 1	Topic selection and problem identification	Week 1
Phase 2	Literature review and existing-solution analysis	Week 2
Phase 3	Requirement analysis and system design	Week 3
Phase 4	DNA dataset collection and preprocessing	Week 4
Phase 5	Dynamic Programming algorithm development	Week 5
Phase 6	Alignment and traceback implementation	Week 6
Phase 7	Genome pattern-analysis implementation	Week 7
Phase 8	Visualization and interface development	Week 8
Phase 9	Testing and verification	Week 9
Phase 10	Performance evaluation and result analysis	Week 10

Simple Gantt-Style Representation

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Topic & Problem	✓											
Literature Review		✓										
Requirements & Design			✓									
Dataset Collection				✓								
Algorithm Development					✓	✓						
Pattern Analysis							✓					
Visualization								✓				
Testing									✓			
Results Analysis										✓		
Documentation											✓	
Final Presentation												✓

4.8.2 Student Roles & Contributions

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project “DNA Sequence Alignment and Genome Pattern Analysis Using Dynamic Programming Algorithms” addressed the engineering problem of accurately comparing DNA sequences and identifying important similarities and variations while considering computational constraints. A Dynamic Programming-based approach was developed to systematically align DNA sequences by considering matches, mismatches, and gaps.

The developed system implements the sequence-alignment process through a scoring matrix and traceback mechanism and analyzes the resulting alignment to identify matching regions, mismatches, gaps, conserved regions, and genome patterns. The system was tested using different DNA sequence inputs to verify input validation, matrix generation, alignment accuracy, scoring, traceback, and pattern analysis.

The results demonstrate that Dynamic Programming provides a reliable and systematic method for obtaining optimal sequence alignments under the selected scoring scheme. The project also demonstrated the computational limitation of the conventional approach, where processing time and memory requirements increase with sequence length.

Overall, the project successfully achieved its primary objectives by developing a functional software prototype for DNA sequence alignment and genome pattern analysis. It demonstrates the practical application of algorithm design, optimization, data processing, and computational analysis to a real-world bioinformatics problem.

5.2 Future Work

The proposed system can be further improved in the following areas:

1. Large-scale genome processing: Optimize the algorithm to handle much longer DNA sequences and complete genomic datasets.
2. Memory optimization: Implement memory-efficient Dynamic Programming techniques to reduce the $O(mn)$ space requirement.
3. Parallel processing: Use GPU or parallel computing techniques to reduce execution time for large sequence datasets.
4. Advanced pattern detection: Incorporate more sophisticated motif and conserved-region detection algorithms.
5. Multiple sequence alignment: Extend the system from pairwise alignment to multiple DNA sequence alignment.
6. Improved visualization: Develop an interactive graphical interface for viewing scoring matrices,

traceback paths, alignments, and detected patterns.

7. Database integration: Connect the system with publicly available biological sequence databases for automated sequence retrieval.
8. Machine learning integration: Combine sequence alignment with machine-learning techniques for advanced genome pattern classification.
9. Benchmarking: Compare the proposed implementation with established tools such as BLAST and other alignment software using larger datasets.
10. Web-based implementation: Develop a web application so that users can perform sequence analysis through a user-friendly interface.

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

- Learned the fundamentals of DNA structure and nucleotide sequence representation.
- Learned the concepts of DNA sequence alignment and genome pattern analysis.
- Gained knowledge of Dynamic Programming and optimal-substructure concepts.
- Learned how Needleman–Wunsch and Smith–Waterman algorithms perform sequence alignment.
- Understood the role of match, mismatch, and gap scoring.
- Learned how to construct and process a Dynamic Programming scoring matrix.
- Learned the traceback process for obtaining an optimal alignment.
- Gained knowledge of computational complexity, particularly $O(mn)$ time and space requirements.
- Learned how to test and evaluate an algorithm using different input datasets.
- Gained experience in interpreting computational results and performance measurements.

5.3.2 Engineering & Professional Skills Developed

- Algorithm design: Developed an algorithmic solution to a real-world computational problem.
- Programming: Improved Python programming and debugging skills.
- Problem solving: Learned to break a complex sequence-analysis problem into smaller computational tasks.
- Data analysis: Developed skills in processing and interpreting DNA sequence data.
- Testing and validation: Learned to create test cases and verify algorithm outputs.
- Performance evaluation: Learned to analyze execution time and memory requirements.
- Technical documentation: Improved the ability to document engineering methods, results, limitations, and decisions.
- Research skills: Learned to study research papers and compare existing technical approaches.
- Engineering judgment: Developed the ability to select an algorithm based on accuracy, computational cost, and project requirements.

REFERENCES

- [1] S. B. Needleman and C. D. Wunsch, “A general method applicable to the search for similarities in the amino acid sequence of two proteins,” *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, Mar. 1970, doi: 10.1016/0022-2836(70)90057-4.
- [2] T. F. Smith and M. S. Waterman, “Identification of common molecular subsequences,” *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195–197, Mar. 1981, doi: 10.1016/0022-2836(81)90087-5.
- [3] S. Altschul, W. Gish, W. Miller, E. Myers, and D. Lipman, “Basic local alignment search tool,” *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, Oct. 1990, doi: 10.1016/S0022-2836(05)80360-2.
- [4] D. W. Mount, *Bioinformatics: Sequence and Genome Analysis*, 2nd ed. Cold Spring Harbor, NY, USA: Cold Spring Harbor Laboratory Press, 2004.
- [5] P. Compeau and P. Pevzner, *Bioinformatics Algorithms: An Active Learning Approach*, 2nd ed. Active Learning Publishers, 2015.
- [6] R. Durbin, S. R. Eddy, A. Krogh, and G. Mitchison, *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge, U.K.: Cambridge University Press, 1998.
- [7] International Organization for Standardization, *ISO/IEC 25010:2023 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*. Geneva, Switzerland: ISO, 2023.
- [8] International Organization for Standardization, *ISO/IEC 27001:2022 — Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements*. Geneva, Switzerland: ISO, 2022.
- [9] National Center for Biotechnology Information, “NCBI Nucleotide Database.” [Online]. Available: <https://www.ncbi.nlm.nih.gov/nuccore/>. [Accessed: Sep. 2, 2026].
- [10] National Center for Biotechnology Information, “BLAST: Basic Local Alignment Search Tool.” [Online]. Available: <https://blast.ncbi.nlm.nih.gov/>. [Accessed: Sep. 2, 2026].

APPENDICES

Appendix A – Detailed Calculations

This appendix presents the important calculations used in the DNA sequence alignment and genome pattern analysis system.

A.1 Dynamic Programming Matrix Size

For two DNA sequences of lengths m and n , the Dynamic Programming (DP) matrix contains:

$$(m + 1) \times (n + 1)$$

cells.

For example, if:

- Sequence 1 length = 100 bases
- Sequence 2 length = 120 bases

then:

$$(100 + 1) \times (120 + 1) = 101 \times 121 = 12,221$$

DP cells are required.

A.2 Alignment Scoring

The system uses a configurable scoring scheme. An example is:

Alignment condition	Score
Match	+2
Mismatch	-1
Gap	-2

A.3 Dynamic Programming Recurrence

For global alignment, the score at matrix cell $D[i][j]$ is calculated as:

$$D[i][j] = \max \begin{cases} D[i-1][j-1] + S(x_i y_j) \\ D[i-1][j] + Gap \\ D[i][j-1] + Gap \end{cases}$$

where:

- x_i and y_j are bases from the two DNA sequences.
- $S(x_i y_j)$ is the match or mismatch score.
- Gap represents the gap penalty.

A.4 Time and Space Complexity

For sequences of lengths m and n :

$$\begin{aligned} \text{Time Complexity} &= O(mn) \\ \text{Space Complexity} &= O(mn) \end{aligned}$$

for the standard full-matrix implementation.

These calculations explain why computational cost increases as the input sequences become longer.

Appendix B – Engineering Drawings / Schematics

As this project is a software-based bioinformatics system, conventional mechanical or electrical engineering drawings are not applicable.

Instead, the following software architecture diagram should be included:

DNA Sequence Input



Input Validation



Sequence Preprocessing



Alignment Method Selection



Dynamic Programming Matrix



Traceback



Aligned DNA Sequences



Pattern Analysis



Results and Visualization

The schematic demonstrates the flow of data through the proposed system.

Recommended figures for this appendix:

1. Overall system architecture
2. Dynamic Programming matrix structure
3. Needleman–Wunsch alignment flowchart
4. Smith–Waterman alignment flowchart
5. Sequence traceback process
6. Genome pattern-analysis workflow

Appendix C – Source Code / Code Repository Information

The project was implemented using Python and relevant scientific/bioinformatics libraries.

C.1 Programming Environment

Component	Technology
Programming Language	Python
Development Environment	Google Colab / Jupyter Notebook
Numerical Processing	NumPy

C.2 Main Software Modules

The source code is organized into the following logical modules:

1. **Sequence Input Module** – accepts two DNA sequences.
2. **Validation Module** – verifies that the sequences contain valid nucleotide symbols.
3. **Scoring Module** – applies match, mismatch, and gap scores.
4. **DP Matrix Module** – constructs the alignment matrix.
5. **Traceback Module** – determines the optimal alignment.
6. **Pattern Analysis Module** – identifies matches, mismatches, gaps, and recurring/conserved patterns.
7. **Visualization Module** – displays alignment and performance results.
8. **Performance Module** – measures execution time and resource requirements.

C.3 Essential Code Excerpt

Only representative code excerpts should be included in the main report. The complete implementation can be maintained in the approved project repository.

Example of the core Dynamic Programming calculation:

```
for i in range(1, m + 1):
    for j in range(1, n + 1):

        if seq1[i - 1] == seq2[j - 1]:
            diagonal = dp[i - 1][j - 1] + match
        else:
            diagonal = dp[i - 1][j - 1] + mismatch

        up = dp[i - 1][j] + gap
        left = dp[i][j - 1] + gap

        dp[i][j] = max(diagonal, up, left)
```

C.4 Repository Information

Repository: [Insert approved GitHub/GitLab repository link here]

Repository contents may include:

- Python source files
- Jupyter/Google Colab notebooks
- Sample DNA sequences
- Input datasets
- Generated alignment results
- Performance results
- Figures and graphs
- README/documentation
- Requirements/dependency file

The repository should be maintained separately from the main report where permitted by institutional guidelines.

Appendix D – Datasheets**Not Applicable – Software Project**

The project does not involve physical electronic, mechanical, electrical, or laboratory hardware requiring manufacturer datasheets.

Software and library information is documented in Appendix C.

Appendix E – Experimental / Raw Data

This appendix contains the input DNA sequences and raw experimental/performance data used to evaluate the system.

E.1 Sample Input Data

Test Case	Sequence 1	Sequence 2
TC01	ACGTACGT	ACGTCGT
TC02	ATCGATCG	ATCGATCG
TC03	ACGTGGCA	ACGTAGCA
TC04	GATTACA	GACTACA

E.2 Raw Performance Data

The following table can be used to record actual experimental measurements:

Test Case	Seq. 1 Length	Seq. 2 Length	DP Cells	Execution Time	Memory Usage	Alignment Score
TC01	8	7	72	[Actual]	[Actual]	[Actual]
TC02	8	8	81	[Actual]	[Actual]	[Actual]
TC03	8	8	81	[Actual]	[Actual]	[Actual]

E.3 Expected Analysis

The experimental results are expected to demonstrate that:

- Increasing sequence length increases the number of DP matrix cells.
- Highly similar sequences generally produce higher alignment scores.
- Mismatches reduce the alignment score.
- Insertions/deletions introduce gaps.
- Computational requirements increase with sequence length.

Appendix F – Ethics / Institutional Approval

Not Applicable / No Formal Approval Required, provided that the project uses publicly available, synthetic, or non-identifiable DNA sequence data and does not involve human participants, patient samples, clinical diagnosis, or laboratory experimentation.

The project should follow responsible data-handling practices when biological sequence data are obtained from public databases.

If identifiable human genetic data or biological samples are introduced in future work, appropriate institutional ethical approval and data-protection procedures should be obtained before use.

Appendix G – Project Meeting / Progress Records

This appendix contains evidence of project progress and meetings.

Evidence may include:

- Meeting photographs
- Supervisor comments
- Progress screenshots
- Review forms
- Project discussion records
- Presentation/review documents

Appendix H – User / Industry Feedback

Not Applicable, if no external user, industry partner, or professional evaluator was involved.

If feedback was collected, the following format may be used:

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not Applicable, if the project has not resulted in a publication, patent, competition entry, or commercial product.

If applicable, include:

- Research paper/publication details
- Conference presentation
- Patent information
- Competition certificate
- Award/certificate
- Product demonstration
- Software release
- Project exhibition evidence

Appendix J – Individual Contribution Evidence

This appendix is **mandatory for team projects**. It provides evidence of each member's contribution to the project.

J.1 Individual Contribution Table

Team Member	Contribution Area	Work Completed	Evidence
[Member 1]	Literature Review	Reviewed sequence-alignment algorithms and related research	Literature review/document
[Member 2]	Algorithm Development	Implemented Dynamic Programming alignment	Source code
[Member 3]	Pattern Analysis	Developed genome-pattern analysis module	Source code/results

J.2 Evidence of Contribution

Possible evidence includes:

- GitHub/GitLab commit history

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)